Annexe A16 — Les dettes corrigées

LibraryV3 · Phase G, étape 3 (pas 3.1 à 3.5) · 7 au 11 octobre 2026 · ce qu'elles étaient, pourquoi on les a corrigées, ce que cela apporte

Résultat en une ligne. La scène active passe d'environ **750 lignes jaunes** à chaque chargement à **zéro**, et chaque valeur que le moteur choisit à la place de l'auteur est désormais **affichée**. En chemin, on a trouvé et corrigé une vingtaine de défauts **silencieux** : un plantage sur une clé absente, un comportement indéfini, une lumière chargée du mauvais type, des caches qui rendaient le mauvais objet, un vaisseau qui ne déclenchait jamais ses zones de dégâts.

Cette annexe est l'**inventaire**. Chaque dette y est décrite en trois temps : ce qu'elle était, pourquoi la corriger, ce que la correction apporte. Les mécanismes sont détaillés dans les annexes thématiques : A17 (format JSON), A18 (OBJ, MTL et ResourceManager).

1. Le point de départ : un avertissement qui sonne toujours

Au début de l'étape 3, le chargement de la scène active écrivait environ 750 lignes jaunes :

Origine	Nombre	Nature
`[Mesh] cle ignoree 'texture'`	641	clé écrite, jamais lue
`[Trigger] role` et 3 évènements absents	34 + 60	clés lues, jamais écrites
`[Camera] depthDisplayRange`, `lodTolerancePx`	10	idem
divers (Mesh, Health, CameraFollow, Light)	≈ 6	idem

Le jour où un nœud perd sa clé `model`, la seule ligne qui compte se noie dans 750 lignes. **Un avertissement qui sonne toujours finit par ne plus être lu.**

> × Retirer l'avertissement de `Read`. C'est la correction d'une ligne : elle supprime en silence la détection des clés vraiment requises (`model`, `target`, `radius`). On échange le bruit contre l'aveuglement.

> × Tester `Has()` avant chaque lecture (la correction envisagée dans le plan). La clé est écrite deux fois : une faute de frappe dans `Has("lodTolerancePX")` fait que la clé n'est jamais lue, et le contrat reste implicite.

La bonne lecture a été donnée par Onky : **une clé absente est un souci**, que l'oubli soit volontaire ou non. Les 750 lignes n'étaient pas du bruit à faire taire dans le code, mais **750 trous dans les fichiers de scène**. La règle complète est dans A17 § 1.



Figure 1 — Soucis (warnings) au chargement de la scène active, à chaque pas. Le nombre monte avant de tomber à zéro : on rend d'abord visible, puis on corrige les données. Un compteur qui descend sans jamais avoir monté n'a rien prouvé.

2. Les dettes du Serializer et de JsonReader (pas 3.1 à 3.3)

#	Dette	Ce qui se passait	Correction	Ce que cela apporte
1	`Read` ne disait pas quelle valeur il prenait	« Prise en compte de la clé par défaut : » suivi de rien	message unique `… — defaut pris : X` (`WarnDefault`), valeur formatée par `std::format`	chaque souci dit ce que le moteur a fait à la place de l'auteur
2	`ReadVector` silencieux	`translation` absente, `[0, 90]` ou `[1, 2, 3, 4]` acceptés sans un mot	tableau de **exactement** 3 nombres, sinon souci	une rotation mal écrite ne passe plus pour une rotation nulle
3	Pas de forme écrite pour « je délègue au moteur »	la sentinelle `-1` dans la scène	`null` = délégation écrite, annoncée en **info** (blanc, non compté)	« j'ai oublié » et « je le veux » se distinguent enfin
4	Surcharges caméra validées trop tard	`lodTolerancePx: 0` ignoré en silence dans `BuildViewData`, à chaque frame	`ReadPositiveOverride` au parsing ; composant `bool m_hasX` + `float m_X` ; **assertion** dans `BuildViewData`	une donnée d'auteur est validée **une fois, à l'entrée** ; le chemin chaud ne fait que vérifier l'invariant
5	`engine.json` : `depthDisplayRange` jamais validé	`0` → mode Depth tout noir, négatif → tout blanc, sans message	même garde que `tolerancePx` : `!(v > 0)` ou `!isfinite(v)`	une ligne de config ne se déguise plus en bug de rasterizer
6	`sceneName` absent	`operator[]` insère un `null`, `get<string>()` lève une exception non rattrapée : **le programme se termine**	lecture par `JsonReader`	un souci au lieu d'un plantage
7	`id` absent	`operator[]` sur un json **const** : comportement indéfini (R28)	`Read("id")`, refus franc si vide	plus d'UB ; un nœud sans nom est refusé
8	Aucun `WarnUnread` au niveau nœud	`"parnet": "Earth"` faisait du nœud une **racine**, sans un mot	chaque nœud lu par **un seul** lecteur, d'un bout à l'autre ; `Delegate` pour `parent` et `components`	une faute de frappe dans la hiérarchie est signalée
9	`parent` absent = racine en silence	7 racines implicites	D1 : `"parent": null` écrit	une racine est une intention, pas un oubli
10	Le tableau `nodes` lu deux fois en brut	la passe 2 relisait le JSON pour `parent`	`parent` **lu** en passe 1, **résolu** en passe 2 (`ResolveParents`)	un enfant peut précéder son parent (R26) ; plus de seconde lecture

#	Dette	Ce qui se passait	Correction	Ce que cela apporte
11	Blocs absents silencieux	`if (r.Has("lod"))` dans `EngineConfig` et `ConfigManager` ; `Child()` rendait un lecteur vide sans rien dire	`Child` : absent → souci ; `null` → lecteur **délégué** (ses clés manquantes sont annoncées en info, « bloc null ») ; mauvais type → souci	D8 : un bloc manquant donne la liste complète des valeurs prises
12	`far` lu même avec `infiniteFar: true`	impossible d'obtenir zéro souci : écrire `far` donnait « ignoré », ne pas l'écrire donnait « absente »	chaque branche lit **ses** clés (même règle que la lentille)	une caméra correctement écrite se charge sans souci
13	Gizmo : deux défauts pour le même cas	bloc absent → 0 ; bloc sans `length` → 2,0	`Child("gizmo")` ; `length` par défaut 0 ; `null` = « pas de gizmo » annoncé	un seul défaut, une seule signification
14	Clé de scène `m_currentHealth`	un nom de membre C++ avait fui dans le format ; défaut fixe 100, même si `maxHealth` vaut 80	D6 : `currentHealth`, défaut = `maxHealth`	le format ne dépend plus de l'implémentation
15	Le Soleil chargé en lumière **Ambient**	`"POINT_LIGHT"` inconnu de `ParseLight` ; le message existait mais était noyé dans 750 lignes	D4 : `"Point"`	le premier vrai bug trouvé grâce au silence retrouvé
16	641 `texture` jamais lues	une texture n'est pas une propriété d'instance	D3 : clé supprimée ; la texture passe par le MTL (A18)	− 641 lignes, et l'information va au bon niveau
17	`config.json` : bloc `configuration` lu, absent, **inutilisé**	3 soucis à chaque lancement pour `REP_OBJ_DEFAULT` / `REP_GFX_DEFAULT`, restes de l'ancien moteur (chemins `G:\`)	lecture et champs **supprimés**	une configuration que personne n'utilise est une dette, pas une donnée

> **À retenir.** On valide une donnée d'auteur **une seule fois, à son entrée dans le moteur**, et on établit un invariant. Le chemin chaud ne corrige jamais : il **asserte**. Valider `lodTolerancePx` dans `BuildViewData` aurait écrit 1 680 lignes par seconde (1 message × 4 vues × 420 fps), au mauvais endroit, sans connaître le nœud fautif.

Deux régressions ont été attrapées en cours de route **par la règle elle-même** : la ligne `near` supprimée par erreur avec l'ancienne lecture de `depthDisplayRange` (signalée par `WarnUnread` comme « clé ignorée 'near' » sur les 5 caméras), et un `WarnDefault(key, "absente")` oublié dans `Read` (détecté par l'assertion du test D3.3b). Sans `WarnUnread` et sans compteur, ces deux erreurs seraient passées.

3. Les dettes des matériaux (pas 3.3e)

#	Dette	Ce qui se passait	Correction	Apport
18	MTL introuvable	`if (!file.is_open()) return result;` : toutes les faces perdaient leur matériau en silence	souci `[MTL] fichier introuvable`	18 références cassées révélées (`planet.mtl` n'existait pas)
19	`usemtl` inconnu	les faces recevaient `MaterialHandle::Invalid()` sans message	souci `[OBJ] usemtl '…' introuvable`, **un par nom**, pas un par face	18 de plus

#	Dette	Ce qui se passait	Correction	Apport
20	Directives MTL ignorées	`map_Bump` (Blender, `gen_ship.py`) et `map_Ke` (réacteurs du vaisseau) disparaissaient	prises en charge ; toute autre directive inconnue est signalée avec `fichier:ligne`	le MTL du vaisseau est lu en entier
21	Textures référencées mais jamais vérifiées	aucun chargeur avant L09	`LoadTexture` vérifie l'existence : absente → souci, présente → info	6 chemins faux du vaisseau trouvés **avant** L09
22	Générateurs en retard sur les données	`gen_sphere_lod.py` écrivait encore `planet.mtl` ; `gen_ship.py` des chemins faux et des UV qui se recouvraient (124 triangles)	une seule définition des matériaux dans `gen_meshes.py` ; `gen_ship.py` corrigé (5 valeurs)	tous les générateurs reproduisent le dépôt **octet par octet**

Le contrôle positif de ce pas : **avec le code seul**, la scène active affichait 42 soucis jusque-là muets ; **avec les données corrigées**, 0. Voir A18 § 3.

4. Les dettes du ResourceManager (pas 3.4)

Les quatre ont la même racine : **la clé d'un cache ne contenait pas tout ce qui détermine le résultat.**

#	Cache	Clé avant	Ce qui se passait	Clé après
23	meshes (`LoadMeshChecked`)	chemin canonique	le même `.obj` demandé avec `scale = 2` rendait le mesh à l'échelle 1, sans un mot	chemin + `OBJLoadOptions::CacheKey()`
24	chaînes de LOD	chemin canonique	même défaut, pour toute la chaîne	chemin + options
25	matériaux	**nom seul**	deux MTL définissant `Material` (défaut de Blender) : le second objet recevait le matériau du premier	`<MTL canonique>#<nom>`
26	bibliothèques MTL	**aucun cache**	un MTL relu à chaque niveau de LOD (4 fois par chaîne) ; textures résolues depuis le dossier de l'**OBJ**	cache par chemin canonique du MTL ; textures relatives au **MTL**

Ce que cela apporte : 18 → 9 lignes `[MTL]` au chargement, et la garantie qu'un objet demandé est **l'**objet obtenu. Détails dans A18 § 4.

5. Les dettes trouvées en chemin (pas 3.5)

#	Dette	Correction	Apport
27	Faux avertissement `[Prof] budget de frames epuise (0)` à chaque lancement sans `--bench`	`s_armed`, posé par `Profiler::Begin` ; `BeginFrame` ne mesure rien si le harnais n'est pas armé	un instrument ne ment plus sur son propre état ; `warnCount` n'est plus pollué
28	`.gitignore` : `*.obj` (modèle Visual Studio, prévu pour les fichiers objets du compilateur) excluait les **maillages**	`!assets/Meshes/*.obj`	un clone du dépôt peut charger les scènes

#	Dette	Correction	Apport
29	Scènes de test cassées : `cube.obj` et `sphere 10 faces.obj` absents du dépôt, MTL 3ds Max avec `Tf` et un chemin absolu `C:\…\fishy.pcx`	meshes versionnés, nom en minuscules, MTL nettoyés	4 scènes de test à 0 souci
30	`solar_system_shuffled.json` (test R26) avait **dérivé** de sa référence : 9 valeurs différentes	reconstruit à partir de `scene_test.json`	le test « même hiérarchie quel que soit l'ordre » prouve de nouveau quelque chose
31	`Player_Ship` en `role: "zone"`	`"probe"`, dans les scènes du système solaire et dans `scene_test.json`	`TriggerSystem` ignore les paires zone-zone : **le vaisseau ne déclenchait jamais aucune zone de dégâts**. Vrai bug de gameplay
32	Évènements de trigger hors vocabulaire (`ENTER_SUN_CORONA` …) dans les scènes v1	traduits par le générateur : zones inoffensives → `""`, zones de danger → `STARTED_TAKING_DAMAGE` / `TAKING_DAMAGE` / `STOPPED_TAKING_DAMAGE`	les zones de danger agissent réellement

> × Garder une copie de référence maintenue à la main (dette 30). Une copie **dérive**, toujours. Le fichier mélangé doit être **produit** depuis sa référence, ou le test doit mélanger lui-même en mémoire.

6. Un piège rencontré pendant les tests : le pointeur pendant

En ajoutant au test `TestLodChain` une étape qui charge une 4ᵉ chaîne, on a cassé l'étape suivante (« L0 partagé avec rock_a.obj »). La cause n'était pas le chemin du fichier, mais un **pointeur pendant** : le test gardait `const LodChain* a = rm.GetLodChain(h[0])`, et le `push_back` de la nouvelle chaîne a fait **réallouer** `m_lodChains`. En Debug, MSVC remplit la mémoire libérée de `0xDD` : le handle lu valait `0xDDDDDDDD`.

> × Garder un `const LodChain*` à travers un appel qui peut enregistrer une chaîne. `ResourceManager.h` le dit pourtant : pointeur **temporaire**, valable jusqu'au prochain `RegisterLodChain`. On garde le **handle** et on relit le pointeur au moment de s'en servir.

7. Les tests ajoutés

Test	Ce qu'il verrouille
D3.1	les quatre écritures d'une clé (exacte, absente, `null`, type invalide), avec contrôle positif de `WarnUnread`
D3.2	les surcharges caméra : valeur, `null`, absente, hors domaine (0, négatif, `1e39` → `inf`)
D3.3a	niveau nœud : enfant avant parent, `sceneName` absent, faute `parnet`, 4 refus francs (5 lignes rouges volontaires)
D3.3b	blocs : présent, `null`, absent, mauvais type
D3.3c	`far` conditionnel, gizmo, `currentHealth`
D3.3e	MTL/OBJ : mtllib absent, usemtl inconnu, directives, textures
D3.4a / b / c	clés des caches : options, MTL + nom, lecture unique des MTL

`Logger::warnCount()` est l'outil commun : **un silence ne se constate pas à l'œil, il se compte.** Chaque test qui affirme « aucun souci » contient aussi un cas où le compteur **doit** bouger, pour prouver qu'il n'est pas débranché.

8. Ce qui reste ouvert

Point	Où	Quand
nlohmann convertit `2.7` en `int` 2 et `-1` en `uint32_t` 4 294 967 295 sans erreur (`LodChainLoader` lit `vertices` et `faces` en `uint32_t`)	`JsonReader`	à traiter
`solar_system_v2.json` : 432 composants inconnus (Body, Orbit, Spin…), schéma cible d'un futur Serializer	LIB	futur
Surcharge de matériau par instance (`"material": "M_Earth"`)	Serializer, L08	L08
Chargement réel des textures, cache de textures, mipmaps	ResourceManager, L09	L09
Test R26 automatique (mélange en mémoire) au lieu d'un fichier mélangé	Tests	à l'occasion

9. Aide-mémoire

Règle	Énoncé
Règle d'Onky (7 oct. 2026)	toute clé lue est **écrite** : valeur exacte (silence), `null` (info : valeur retenue), absente (souci : défaut pris)
R28	`find()`, jamais `operator[]` sur un json (insertion ou UB)
Préfixe `_`	clé assumée, non lue : documentation (`_type`, `_note`, `_range`)
Validation	une fois à l'entrée ; ensuite on asserte l'invariant
Clé de cache	contient **tout** ce qui change le résultat
Compteur	un test « zéro souci » contient un contrôle positif